

@msbc React Toolkit

Internal Frontend Platform — Architecture & Developer Guide

Prepared: 15 July 2026 · Scope: full monorepo (6 packages + dev app) · Audience: engineers onboarding to the toolkit

1. Executive Summary

The **@msbc React Toolkit** is our internal frontend platform: a single pnpm monorepo that ships six versioned npm packages plus a development harness app. Together they let product teams build data-driven screens — forms, dashboards, tables, import flows — by writing **JSON configuration instead of hand-coding UI**, while still guaranteeing a consistent look, consistent API/auth behaviour, and consistent validation across every application that consumes them.

The packages are deliberately layered. At the bottom are tiny, dependency-free helpers; above them a networking layer and a themed component library; above those a configuration-driven orchestration layer; and at the top ready-made application scaffolding (routing shell and an import wizard). Each layer only depends on the layers beneath it, which keeps the build a clean directed graph and makes each package independently publishable and testable.

This document explains, package by package, **what each piece does, what calls what, and why the toolkit exists** — the concrete engineering benefits it delivers over building each app from scratch.

Package	Version	Role in one line	Size (src)
@msbc/utils	1.0.2	Tiny shared helpers: BEM class names, isEmpty, debounce hooks, colour contrast	~174 LOC
@msbc/data-layer	1.0.8	Networking backbone: axios client, token/refresh manager, useApiRequest hook, Redux slice generator	~626 LOC
@msbc/react-toolkit	1.1.17	Themed, unstyled-primitive component library (19 components) wrapping best-in-class libs	~6,500 LOC
@msbc/config-ui	1.2.7	JSON-driven ConfigurableForm and ConfigurableDashboard built on the two layers above	~2,400 LOC
@msbc/config-app-shell	1.0.3	Config-driven routing + layout frame (sidebar/header/content)	~124 LOC
@msbc/import-utils	1.0.4	Ready-made multi-step CSV/Excel Import Wizard	~556 LOC
app/dev-app	(private)	Vite harness that consumes every package end-to-end	—

2. Why This Toolkit Exists — The Benefits

Before the toolkit, every new screen meant re-solving the same problems: wiring axios, handling 401s and token refresh, restyling a date picker, re-implementing form validation, rebuilding a paginated table. The toolkit exists to solve each of those **once**, correctly, and then reuse the solution everywhere. The concrete benefits:

2.1 Consistency by construction

- **One design system.** Every component reads its colours, spacing, radius and typography from CSS custom properties (design tokens). Swapping a brand theme (e.g. `Journies`) restyles every app at once — no per-app CSS edits.
- **One class-naming convention.** All components emit BEM class names through a single `classname()` helper from `@msbc/utils`, so markup and SCSS stay predictable and collision-free.
- **One API contract.** Every network call — whether from a hook, a Redux slice, a Dropdown, or a Dashboard — goes through the same axios instance, so auth headers, token refresh, timeouts and error shapes are identical everywhere.

2.2 Development velocity

- **Config over code.** A full CRUD dashboard or a validated multi-field form is expressed as a JSON object, not hundreds of lines of JSX. Product teams describe **what** they want; `config-ui` handles **how** it renders.
- **Batteries included.** Complex third-party libraries (AG-Grid tables, react-select dropdowns, FilePond uploads, Google Maps, react-datepicker) are already integrated, themed, and made API-aware — teams never touch them directly.
- **Ready-made flows.** The Import Wizard and App Shell mean common app-level plumbing (CSV import, routing + layout) is a drop-in, not a rebuild.

2.3 Correctness & maintainability

- **Auth/refresh handled centrally.** The 401 → refresh-token → single-retry cycle lives in one axios interceptor. Fix it once, every app benefits; no app can get it subtly wrong.
- **Validation is declarative.** Form rules (required, min/max, pattern, conditional `requiredIf`, custom functions) are described in config and mapped to react-hook-form in one place, so validation behaves identically across forms.
- **Independent versioning.** Each package is semver-versioned and published separately; apps upgrade on their own schedule. `workspace:*` links are rewritten to pinned versions at release time.
- **Documented & testable in isolation.** Storybook is the living playground/spec for react-toolkit; it is even shipped as a static site via Docker/nginx.

2.4 Separation of concerns

The layering means each package has exactly one job. A bug in networking is fixed in `data-layer`; a visual tweak lives in `react-toolkit`; a new field type is a `config-ui` concern. Teams can reason about, test, and release each layer independently.

3. The Monorepo at a Glance

The repo is a **pnpm workspace** (`pnpm-workspace.yaml` globs `packages/*` and `app/*`). pnpm links the packages together locally via the `workspace:*` protocol, so editing a package is instantly reflected in its consumers without publishing.

3.1 Dependency graph (who imports whom)

Internal `@msbc/*` imports form a clean, acyclic layering. Arrows mean “depends on”:

```
app/dev-app
|
+--> @msbc/config-app-shell (react-router-dom)
+--> @msbc/import-utils -----+
+--> @msbc/config-ui -----+----+
+--> @msbc/react-toolkit -----+----+
+--> @msbc/data-layer -----+----+
                                   |
@msbc/import-utils --> config-ui, data-layer, react-toolkit
@msbc/config-ui    --> react-toolkit, data-layer, utils
@msbc/react-toolkit --> data-layer, utils
@msbc/data-layer   --> (axios, @reduxjs/toolkit only) [no internal deps]
@msbc/utils        --> (nothing internal) [the base]
```

Empirically confirmed import counts across the codebase: `@msbc/data-layer` is imported 14 times, `@msbc/utils` 11, `@msbc/react-toolkit` 8, `@msbc/config-ui` 4 — exactly matching the layering (the lower a layer, the more widely it is depended upon).

3.2 Build & type strategy

- Most packages build with **tsup** (esbuild) producing **dual CJS + ESM** bundles plus `.d.ts` types. `react-toolkit` adds the `esbuild-sass-plugin` to compile its SCSS into a shipped `dist/index.css`.
- `data-layer` builds with plain `tsc -b`.
- `react`, `react-dom`, and the internal `@msbc/*` packages are marked **external / peerDependencies** so consuming apps never bundle duplicate copies of React.
- Each package exposes `main` (CJS), `module` (ESM) and `types` in its `package.json exports` map; CSS is exposed as a side-effect import (`./dist/index.css`).

3.3 Repository conventions

- **Package manager:** pnpm (required). `pnpm install`, then `pnpm dev` runs the dev-app.
- **Linting:** flat ESLint config at the root (TypeScript + React + react-hooks rules).
- **Formatting:** Prettier — single quotes, semicolons, 100-char width, 2-space indent, ES5 trailing commas.
- **Branching:** `main` (production) ← `develop` (integration) ← `feature/*`. New components go in a feature branch off `develop` via PR.

tokenManager.ts — token storage & refresh

- `createTokenManager(opts?)` returns a closure holding tokens. Storage defaults to `sessionStorage`, falling back to an in-memory `Map`-backed `Storage` shim for SSR/tests/Safari-private-mode.
- Methods: `setTokens`, `getAccessToken`, `getRefreshToken`, `clearTokens`, `isAuthenticated`, `refreshAccessToken`, `getAllTokens`, `setRefreshEndpoint`. A `defaultTokenManager` singleton is created and its bound methods are re-exported.
- `refreshAccessToken()` POSTs the refresh token to the configured refresh endpoint, stores the new access token on success, and (in a browser, on 401) can redirect to a login URL.

axiosInstance.ts — the interceptors (the important part)

`createAxiosInstance({ baseUrl, timeoutMs })` builds an axios instance (default timeout 120 s) and installs two interceptors:

- **Request interceptor:** unless the request opts out with `config.authRequired === false`, it reads the access token from `tokenManager` and sets `Authorization: Bearer <token>`. Auth is **on by default**.
- **Response interceptor:** on a `401` for an auth-required request that hasn't already been retried, it marks `_retry`, calls `refreshAccessToken()`, and on success updates the header and **replays the original request once**. If refresh fails, it clears tokens and throws.
- All errors are normalised to a uniform `{ status, message, data }` shape (`normalizeError`) before they propagate.

Design intent The 401 → refresh → single-retry dance and error normalisation live here and only here. Every consumer — hooks, slices, and even the API-aware Dropdown/Table — inherits correct auth behaviour for free and can never implement it inconsistently.

apiClient.ts — typed verb wrapper

Exposes `apiClient` with generic `get / post / put / patch / delete` methods over the singleton axios instance. Each method returns `res.data` (unwraps the axios envelope). `delete` packs its body into `config.data` because axios can't take a delete body positionally.

useApiRequest.ts — the React hook (primary consumption surface)

The hook most app code uses. Options and return shape:

```
const { data, loading, apiError, execute } = useApiRequest({
  url:      '/api/v1/users/',
  method:   'get' | 'post' | 'put' | 'patch' | 'delete', // default 'get'
  params?, body?, config?,                             // config carries axios flags incl. { authRequired }
  autoFetch?: boolean,                                  // default true; GET auto-runs once on mount
  isMultipart?: boolean,                                // FormData mode (skips body merge, sets header)
});
```

- For a GET with `autoFetch`, `loading` starts `true` (no empty-state flash) and the request fires **once** (guarded against React StrictMode double-invoke).
- `execute(override?)` runs the request imperatively and accepts an override `{ url, method, params, body, config }` — so one hook instance can be reused for a dynamic call (e.g. an edit URL built at click time). It returns the response and also **re-throws** on error so callers can `try/catch`.
- The hook itself does no token/retry logic — that is entirely the interceptors' job. It just surfaces the normalised `apiError` and toggles `loading`.

createApiSlice.ts — Redux Toolkit slice generator

For teams that prefer global state, `createApiSlice(name, endpoint, method, configParams?)` generates a standard RTK slice in one call: a `createAsyncThunk ( <name>/fetch )` that calls the same `apiClient` verbs, plus a `createSlice` whose `extraReducers` track `{ data, loading, error }` across pending/fulfilled/rejected. Returns `{ reducer, actions: { fetchData } }`. Because it reuses `apiClient`, Redux users get identical auth/refresh/error semantics to hook users.

4.3 @msbc/react-toolkit — the component library

The largest package (~6,500 LOC, 19 components). Its `index.ts` is a barrel re-exporting every component folder. The core philosophy: **wrap best-in-class third-party libraries, theme them with our design tokens, and make them API-aware.**

Folder convention

Each component lives in its own folder with a consistent shape: `Component.tsx` (implementation), `Component.types.ts` (props), `Component.scss` (BEM styles reading only CSS variables), `index.ts` (barrel), and usually `Component.stories.tsx` (Storybook). 12 of the 19 components ship stories.

The shared field shell: FormFieldWrapper

`FormFieldWrapper` is the single presentational wrapper every form input reuses. It renders (in order) an optional label with a required `*`, the control container, optional help text, and an error block. Its five props — `label, helpText, isRequired, error, isDisabled` — are the standard “field contract”. Confirmed consumers: Input, Dropdown, DatePicker, RangeDatePicker, FileUpload, CheckboxGroup, RadioGroup, List, MultipleItem. This is why every form field in every app looks and behaves the same.

What each component wraps (the “why we use this” story)

Component	Wraps / built on	Notes
Dropdown	react-select (+ Creatable)	Custom auto-scroll MenuList; <code>isCreatable</code> toggle; API-aware.
DatePicker / RangeDatePicker	react-datepicker + react-select	Custom month/year header built from two selects; range variant.
FileUpload	react-filepond + 6 FilePond plugins	Image preview, EXIF orientation, type/size validation, custom file-type icons.
Table	ag-grid-react + ag-grid-community	Themed AG-Grid (themeQuartz wired to CSS vars); generic Table<T>; API-aware.
Map	@react-google-maps/api	Autocomplete + marker drag with reverse-geocoding; address key mapping.
Badge	chroma-js (via @msbc/utils)	Auto-picks readable text colour for any background.
FilterModal / FilterChip	toolkit's own components	Compose Modal/Input/Dropdown/Badge into filter UI.
Input, Button, Checkbox(Group), RadioGroup, ToggleButton, Modal, List, TabsNav, MultipleItem	hand-rolled HTML + SCSS	Native, accessible primitives (TabsNav has full roving-tabindex keyboard nav).

API-aware components

Five components can either take static data **or** an `api` prop and fetch their own options/rows via `useApiRequest` from `data-layer`: **Table, Dropdown, CheckboxGroup, RadioGroup, List**. They share a pattern: `labelKey` / `valueKey` map arbitrary API objects into `{label, value}`, and an optional `apiResponseMapper` normalises the response shape. This is what lets a config say “this dropdown's options come from `/api/roles`” with no extra code.

Theming / design tokens

- Components emit BEM classes via `classname` from `@msbc/utils`; SCSS styles those selectors using **only CSS custom properties** (e.g. `background: var(--color-primary-07)`, `gap: var(--space-100)`).
- The default token set + reset lives in `styles/main.scss` (palettes, semantic colours, radii, a typography scale, spacing). Light/dark are toggled via `.theme-light` / `.theme-dark` (or `:root.dark`).
- Per-brand overrides live in `themes/journies-tokens.css`, referenced by a `themes` registry (`{ Journies: { url: 'journies-tokens.css' } }`). Swap the theme file → restyle everything.

Known token drift (tech debt) `main.scss` uses names like `--color-primary-07` / `--space-100`, while `journies-tokens.css` uses `--primary-07` / `--spacing-md`. The two token files are only partially aligned, and `styles/_tokens.scss`, `_variables.scss`, `_mixins.scss` are currently empty placeholders. Worth reconciling.

4.4 @msbc/config-ui — the JSON-driven orchestration layer

The heart of the “config over code” story. It exposes exactly two features, both driven by a JSON `config` object: **ConfigurableForm** and **ConfigurableDashboard**. It consumes react-toolkit (render targets), data-layer (fetching), and utils (helpers).

ConfigurableForm — config → rendered, validated form

Flow, end to end:

1. The developer passes a `config` (a `JSONFormSchema`) describing `layout`, and either flat `fields` or nested `sections` (which may be groups or **repeatable** sections).
2. `computedDefaults` walks the config to build initial values (respecting `defaultValues` prop → field `default` → type-appropriate empty).
3. A single **react-hook-form** `useForm` instance is created (`mode: 'onChange'`). Its methods are exposed via a ref (`reset`, `setValue`, `getValues`, `trigger`, `setError`, `clearErrors`, `submit`).
4. The tree is rendered: sections/groups/repeatables are dispatched by type guards; each field becomes a `FormElement`.
5. `FormElement` registers the field with `useController`, attaches validation from `mapValidationToRHF`, evaluates `visibleIf` rules, handles `depends` (dependent selects, value population, dependent API URLs), and renders the matching react-toolkit component.
6. Submit goes through `handleSubmit(onSubmit)` (native form submit or the imperative `ref.submit()`).

The field-type registry

`FormComponentRegistry` maps config `type` strings to react-toolkit components:

config type	react-toolkit component
text, email, number, password, textarea, tel	Input (variant selects the HTML input type)
select	Dropdown
radio	RadioGroup
checkbox	CheckboxGroup
list	List
fileUpload	FileUpload
map	Map
date	DatePicker
date-range	RangeDatePicker
custom	your own render fn / component (escape hatch)

Declarative validation

`mapValidationToRHF` translates a field's `validation` config into react-hook-form rules: `required` / `requiredIf` / `requiredIfAny` (conditional on another field), `minLength` / `maxLength` / `min` / `max`, `pattern` (regex), and `custom` (named function from a `customValidators` map). Conditional rules understand repeatable-row-relative paths, so validation works inside dynamic row arrays.

ConfigurableDashboard — config → data table / card view

A full data screen from one config object. Behaviour:

- **Fetching:** uses `useApiRequest` (from data-layer) with `autoFetch: false`; a single effect re-fetches whenever `[debouncedSearch, filters, sort, pagination]` change, composing request params (search, filter values, sort field/direction, and pagination keys — all key names configurable).
- **Rendering:** `mode === 'table'` renders the AG-Grid-backed `Table`; `mode === 'list'` renders `List` with a supplied `CardComponent`. A `ToggleButton` switches modes when `enableModeSwitch` is on.
- **Sorting** comes from AG-Grid's `onSortChanged` (read via the grid API); **pagination** is a separate `Pagination` component (shown only when the response supplies page metadata via `apiResponseMapper`).
- **Filtering:** per-field basic filters (select/date/date-range/text) plus an advanced `FilterModal` that groups filters into `in` clauses.
- **Imperative ref:** `ConfigurableDashboardHandle.handleRefresh(hardRefresh?)` — a soft re-fetch, or (hard) reset of sort/filters/selection then re-fetch. This is how a parent (e.g. after saving a modal) forces the table to reload.

4.5 @msbc/config-app-shell — routing + layout frame

A tiny (~124 LOC) config-driven application frame built on **react-router-dom**. It turns a plain routes array into a rendered app with a persistent sidebar/header/content layout.

- **`AppShell`** (turnkey): renders its own **BrowserRouter**, wraps everything in the internal **AppLayout**, and maps a **routes** array into **<Route>** elements. Each route references a component **by string key**, resolved against a **components: Record<string, FC>** map — which is exactly what lets the route table live in a JSON file. Missing components degrade to a friendly “not found” node; each resolved component receives **configPath={route.config}**.
- **`AppLayout`** (internal): the presentational sidebar/main/header/content frame; renders **{children}**.
- **`LayoutRoute`** (à-la-carte): same frame but falls back to react-router's **<Outlet/>** for nested-route composition.
- **`ShellRoutes`** (à-la-carte): the **<Routes>** -mapping logic on its own (adds index-route support), to be composed inside a consumer-supplied **<Router>** + **LayoutRoute**.

So there are two usage styles: the monolithic **AppShell** (used by dev-app) or the composable **LayoutRoute** + **ShellRoutes** for teams that want to own their router tree.

4.6 @msbc/import-utils — the Import Wizard

A ready-made, config-driven **CSV/Excel import** flow — a thin orchestration layer over the other packages. Its only public export is **ImportWizard**; **FieldMapper** is an internal step.

Three steps, driven by **react-form-stepper**:

1. **Module select** — a **ConfigurableForm** picks the target module; optionally fetches a “field reference” note describing required columns.
2. **Upload** — a **ConfigurableForm** file upload; on submit it POSTs the file (multipart) and the backend returns the file's column headers plus required/optional system fields and a suggested mapping.
3. **Field mapping** — **FieldMapper** renders a two-column table (target field → a Dropdown of source columns), pre-seeded from the suggested mapping, validates that all required fields are mapped, then submits an inverted **header_map**.

It uses **config-ui** for the step forms, **data-layer** (**useApiRequest**, incl. multipart) for all three API calls, and **react-toolkit** (**Button**, **Dropdown**) for the mapper UI. Every label, endpoint, request builder, and response mapper is overridable through **ImportWizardConfig**.

4.7 app/dev-app — the consumer / harness

A Vite React app that mounts every **@msbc** package against real config to prove they work together. It is the best worked example of how to consume the toolkit.

- **`main.tsx`** bootstraps React 18 **createRoot** wrapped in the Redux **<Provider store={store}>** (the router lives inside **AppShell**, not here).

- **`App.tsx`** imports `AppShell`, the route table from `config/routes.json`, and a `components` map (`{ Dashboard, Cart, Form, TodoPage, ImportDataPage }`), then renders `<AppShell routes=... components=... header=... sidebar=... />`. This is the canonical JSON-driven routing pattern.
- **Redux/data-layer demo:** `signupSlice.ts` calls `createApiSlice('signup', '.../signup', 'post', { authRequired: false })`; `store/index.ts` registers the reducer; `useSignup.ts` wraps dispatch + selector into a hook — the stateful path. Pages also demo the imperative `useApiRequest` path.
- **Feature pages** exercise the richest surfaces: `Dashboard` / `TodoPage` (ConfigurableDashboard with filters, actions, card mode, advanced filter), `Form` (ConfigurableForm with dependent selects, conditional visibility, custom validation), `Cart` (react-toolkit Modal/Dropdown/DatePicker), `ImportDataPage` (the ImportWizard).

5. End-to-End Flows — What Calls What

5.1 The request lifecycle (auth, refresh, retry)

```
Component / hook
| useApiRequest(...).execute() (or dispatch(slice.actions.fetchData()))
v
apiClient.get/post/...          -> returns res.data
v
axios singleton instance
| [request interceptor] attach 'Authorization: Bearer <accessToken>' (unless authRequired:false)
v
HTTP request -----> server
| [response interceptor]
|   200 -> pass through (res.data unwrapped by apiClient)
|   401 -> refreshAccessToken() -> POST refresh token
|           success -> set new header, REPLAY original request once
|           failure -> clearTokens(), throw
|   other -> normalizeError -> { status, message, data }
v
useApiRequest sets { data | apiError, loading:false } and re-throws on error
```

Key point: components never see tokens or retries. They see `data`, `loading`, `apiError`. Everything else is handled once in the interceptors.

5.2 ConfigurableForm render flow

```
config (JSON) -> computedDefaults -> useForm (react-hook-form)
|
sections / groups / repeatable --dispatch by type guard--> <FormElement> per field
|
FormElement: useController + mapValidationToRHF (rules)
|
| evaluate visibleIf (show/hide)
| handle depends (dependent selects, populate, dependent API url)
| -> FormComponentRegistry: type -> react-toolkit component
|
| <Input> / <Dropdown> / <DatePicker> / ... (each wrapped in FormFieldWrapper)
|
submit -> handleSubmit(onSubmit) (or ref.submit())
```

5.3 ConfigurableDashboard data flow

```
config.api -> useApiRequest (autoFetch:false)
              ^
    effect keyed on [debouncedSearch, filters, sort, pagination] -> execute({ params })
              |
    response -> apiResponseMapper? -> { data, currentPage, totalRecords, totalPage }
              |
    mode === 'table' -> <Table> (AG-Grid) --onSortChanged--> sortState --> re-fetch
    mode === 'list'  -> <List CardComponent=...>
              |
    <Pagination> (separate) --page change--> pagination --> re-fetch
    parent ref.handleRefresh(hard?) --> reset/re-fetch
```

5.4 Worked example: a User Management screen (from the README)

This shows all layers cooperating — the exact pattern teams follow:

1. A page renders `<ConfigurableDashboard config={...} ref={dashboardRef} />` (config-ui). Its `config.api` points at `/api/v1/users/`; ConfigurableDashboard fetches and paginates the list internally via data-layer.
2. Clicking “Add”/“Edit” opens a `Modal` (react-toolkit) housing a `ConfigurableForm` (config-ui) whose JSON schema defines the user fields and validation.
3. On submit, a custom hook built with `useApiRequest` (data-layer) POSTs/PUTs the payload — inheriting auth + error handling automatically.
4. On success, the page calls `dashboardRef.current.handleRefresh(true)` to reset and reload the table so the new/edited row appears.

6. Build, Versioning & Release

- **Build a package:** `pnpm --filter <pkg> build` (root `pnpm build` builds react-toolkit). tsup emits `dist/index.js` (CJS), `dist/index.mjs` (ESM), `dist/index.d.ts`, and (react-toolkit) `dist/index.css`.
- **Local linking:** consuming packages reference each other with `"@msbc/x": "workspace:*"`; pnpm symlinks the local source — changes are picked up without publishing.
- **Release version pinning:** the `predeploy` script `scripts/update-workspace-versions.js` scans every `package.json`, builds a name→version map, and rewrites every `workspace:*` dependency to the real pinned version before publishing. This keeps published packages installable outside the workspace.
- **Storybook as living docs:** `pnpm sb` runs Storybook for react-toolkit. The `Dockerfile` builds the static Storybook (`build-storybook`) in a Node stage and serves it via nginx — so the component catalogue can be deployed as an internal site.

7. How to Add a New Component (Contributing)

1. Branch off `develop`: `git checkout -b feature/my-component`.
2. Create `packages/react-toolkit/src/components/MyComponent/` with `MyComponent.tsx`, `MyComponent.types.ts`, `MyComponent.scss` (style BEM selectors via CSS variables only), `index.ts` (barrel), and `MyComponent.stories.tsx`.
3. If it is a form field, wrap the control in `FormFieldWrapper` and forward `label/helpText/isRequired/error/isDisabled`. If it needs data, accept an optional `api` prop and use `useApiRequest`.
4. Add `export * from './components/MyComponent';` to `react-toolkit/src/index.ts`.
5. To expose it to config-ui forms, register the `type` → component in `FormComponentRegistry` and add its props type to the config-ui field union.
6. Run `pnpm build` and `pnpm lint`, verify in Storybook, then open a PR into `develop`.

8. Known Issues & Tech Debt

Surfaced during this review — worth tracking:

Area	Issue
data-layer / tokenManager	<code>setRefreshEndpoint</code> has a self-assignment (<code>redirectUri = redirectUri</code>) so the outer <code>redirectUri</code> stays undefined; the 401 <code>window.location.replace(redirectUri!)</code> would navigate to <code>undefined</code> .
data-layer / useApiRequest	There is no top-level <code>authRequired</code> option on the hook — it must be set via <code>config: { authRequired: false }</code> (the axios-level flag).
data-layer / axios	The default exported instance is created eagerly with no <code>baseURL</code> ; base URL/timeout must be supplied per request unless an app builds its own instance.
config-ui / FormElement	<code>formComponentRegistry.get()</code> is called but the actual JSX render is a hardcoded switch, so a runtime <code>register()</code> override wouldn't change what renders. Registry is effectively a type-map, not a live render hook.
react-toolkit / theming	Token-name drift between <code>main.scss</code> (<code>--color-primary-07</code>) and <code>journies-tokens.css</code> (<code>--primary-07</code>); the <code>_tokens/_variables/_mixins.scss</code> partials are empty placeholders.
react-toolkit	Stale backup file <code>multipleItem/MultiInput-bkp.tsx</code> ; minor file-naming inconsistencies (<code>MultipleItem.type.ts</code> , <code>Datepicker.*</code> vs <code>DatePicker.tsx</code>).
dev-app routing	<code>routes.json</code> has no index/redirect route, so <code>/</code> renders an empty content area until a sidebar link is clicked.

9. Quick Reference

Import cheat-sheet

```
// networking
import { useApiRequest, createApiSlice, apiClient } from '@msbc/data-layer';
// primitives
import { Button, Input, Modal, Table, Dropdown, DatePicker } from '@msbc/react-toolkit';
// config-driven
import { ConfigurableForm, ConfigurableDashboard } from '@msbc/config-ui';
import type { DashboardConfig, ConfigurableDashboardHandle } from '@msbc/config-ui';
// app frame
import { AppShell, LayoutRoute, ShellRoutes } from '@msbc/config-app-shell';
// import flow
import { ImportWizard } from '@msbc/import-utils';
// helpers
import { classname, cns, isEmpty, useDebounce } from '@msbc/utils';
```

Mental model in one sentence

utils gives everyone the same primitives; **data-layer** gives everyone the same network; **react-toolkit** gives everyone the same look; **config-ui** turns JSON into forms and dashboards on top of those; **config-app-shell** and **import-utils** are ready-made app-level pieces; and **dev-app** proves it all works together.